# Studio Nx

Backend(FastAPI/Python) 45 — change_password()

2026-10-07

# Backend(FastAPI/Python) 45 — `change_password()`

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/backend/app/services/user_service.py` |
| 작업 | P1-09 비밀번호 변경 · 탈퇴 — 비밀번호 변경 · PRD 01 FR-107 |
| 작성 | Claude (AI) · 2026-10-07 · 사용자 검토 |
| 이전 · 다음 | 44 `check_current_password()` · 46 `withdraw()` |

## 1. 하는 일

지금 비밀번호를 확인하고, 새 비밀번호가 규칙에 맞으면 새 해시로 바꿔 저장한다.

| 지금 `Abcdef1!` → 보낸 값 (지금 / 새) | 결과 |
|---|---|
| `Abcdef1!` / `Newpass9#` | 저장 — 이후 `Newpass9#` 로만 로그인 |
| `Abcdef1?` / `Newpass9#` | 400 `WRONG_PASSWORD` "Password is incorrect." (44) |
| `Abcdef1!` / `abcdefgh` | 422 "Check the password requirements." · `details: {"new_password": ["upper", "number", "special"]}` |
| `Abcdef1!` / `Ab1!` | 422 · `["length"]` |
| `Abcdef1!` / `Abcdef1!` 한글 | 422 · `["allowed"]` |
| `Abcdef1!` / `Abcdef1!` (같음) | 422 **"Use a different password."** · `details: {"new_password": "same_as_current"}` |
| 다른 회원이 `Newpass9#` 를 씀 | 상관없이 저장 |

오류일 때는 DB 의 해시가 그대로다.

## 2. 언제 쓰나

| 쓰는 곳 | |
|---|---|
| `PUT /users/me/password` (47) | userInfo.html 비밀번호 칸 3개(지금 · 새 · 확인) → Change password (P1-10) |

### 사용자 결정 (2026-10-07)

**새 비밀번호가 지금 비밀번호와 같으면 막는다.** 템플릿에는 "(선택)" 이었다.

- 이것은 **같은 회원 안**의 비교다. 다른 회원과 비밀번호가 겹치는지는 검사하지 않는다 — 겹쳐도 문제가 없고, 검사하면 "누군가 이 비밀번호를 쓴다" 는 단서가 된다. 해시마다 소금(salt)이 달라 비교할 수도 없다.
- 예전에 쓰던 비밀번호들까지 막는 것(이력)은 범위 밖.

## 3. 코드

**import** (파일 맨 위)

```
from app.core.security import check_password_rules, hash_password, verify_password
```

**`check_password_rules` · `hash_password` 를 이번에 추가했다** (44 의 `verify_password` 줄에).

**함수**

```
def change_password(db: Session, user: User, current_password: str, new_password: str) -> None:
    check_current_password(user, current_password)
    failed = check_password_rules(new_password)
    if failed:
        raise AppError(
            ErrorCode.VALIDATION_ERROR,
            422,
            "Check the password requirements.",
            details={"new_password": failed},
        )
    if verify_password(new_password, user.password_hash):
        raise AppError(
            ErrorCode.VALIDATION_ERROR,
            422,
            "Use a different password.",
            details={"new_password": "same_as_current"},
        )
    user.password_hash = hash_password(new_password)
    db.commit()
```

### 문법 풀이

| import 문법 | 뜻 |
|---|---|
| `check_password_rules` · `hash_password` | P1-04 (14 · 15) — 가입과 같은 규칙 · 해시 |

| 식 | 뜻 |
|---|---|
| `if failed:` | 빈 목록이면 거짓 — 실패한 조건이 하나라도 있으면 참 |
| `verify_password(new_password, user.password_hash)` | 새 비밀번호를 **지금 해시**에 대 봄 — 맞으면 지금 비밀번호와 같다는 뜻 |
| `-> None` | 돌려줄 것 없음. 라우터는 204 (본문 없음) |

## 4. 흐름

1. 지금 비밀번호 확인 → 틀리면 400.
2. 새 비밀번호 규칙 → 틀리면 422 + 실패한 조건.
3. 지금과 같으면 422.
4. 새 해시로 저장.

## 5. 왜 이렇게

| 선택 | 이유 |
|---|---|
| 지금 비밀번호를 **먼저** | 남이 내 자리에서 새 비밀번호 규칙을 시험해 볼 수도 없게, 본인 확인이 가장 먼저 |

| 선택 | 이유 |
|---|---|
| 규칙 검사를 같음 검사보다 먼저 | 지금 비밀번호는 규칙을 통과했던 값이라 순서가 결과를 바꾸지 않는다. DB 를 보지 않는 가벼운 검사를 앞에 (P1-04 와 같은 원칙) |
| 같음 비교는 `verify_password` | 지금 비밀번호 원문은 서버에 없다 — 해시에 대 봐야 안다. 지금 비밀번호 칸 값과 글자 비교(`current == new`)도 되지만, 해시와 비교해야 "실제로 저장된 것과 같음" 이 확실 |
| `details` 칸 이름 `new_password` | 화면(P1-10)이 그 칸 아래에 문구를 붙일 수 있게 — 요청 본문의 칸 이름과 같음 |
| `refresh` 하지 않음 | 돌려줄 정보가 없다 (응답 204) |

**알아 둘 점 — 다른 기기:** 비밀번호를 바꿔도 다른 브라우저에 이미 있는 리프레시 쿠키(최대 3일)는 그대로 동작한다. 리프레시 토큰을 DB 에 두지 않는 설계라서다 (P1-05 34 §5 와 같은 한계). 필요해지면 "이 시각 이전에 만든 토큰은 무효" 를 회원 행에 두는 방법이 있다 (토큰의 `iat`, P1-05 26).

## 6. 시험 (`tests/test_user_service.py` 에 7개 추가 · 모두 70개, 실제 MySQL)

| 시험 | 확인한 것 |
|---|---|
| 바꾸기 | DB 해시가 바뀜, 새 비밀번호는 맞고 옛 비밀번호는 틀림 |
| 지금 비밀번호 틀림 | 400 `WRONG_PASSWORD`, 해시 그대로 |
| 규칙 3가지 | 약함 · 짧음 · 한글 공백 → 422, `details.new_password` 가 실패 목록 |
| 지금과 같음 | 422 "Use a different password.", `same_as_current`, 해시 그대로 |
| 다른 회원과 같은 비밀번호 | 저장됨 |

전체 시험 328개 통과, `ruff` 검사 통과.